Mukazhan Mumina

SE-2530

BST and Hash Table Analysis

Github link: https://github.com/muminamukazhan/bonuspoint

1.What I did:

I compared two data structures like a Binary Search Tree (BST) and a Hash Table. Both of them support three main operations like insertion, searching, and deletion. But they organize and access data differently.A Binary Search Tree for organizing elements by their keys and a Hash Table uses a hash function.Its to calculate where elements should be stored.

Also I tested three dataset sizes: 100, 1,000, and 10,000 elements. Two types of input data were used:

Random data: Keys were shuffled before insertion.

Sorted data: Keys were inserted in ascending order.

I used Benchmark class to measure execution time and count operation steps.Also I used a IntMap interface to test both implementations.And I created a Check class to test correctness by comparing.

2. Time Complexity:

	BST best	BST worst	Hash average	Hash worst
Insert	O(1)	O(n)	O(1)	O(n)
Search	O(1)	O(n)	O(1)	O(n)
Delete	O(1)	O(n)	O(1)	O(n)

Explanation:

BST

Best case: O(1). The operation can finish right away, for example when the searched key is at the root.

Worst case: O(n). If keys are inserted in sorted order, the tree becomes unbalanced like a linked list, so an operation may have to visit up to n nodes.

Hash Table

Average case: O(1). With a good hash function and a controlled load factor, a key can usually be found after only a few probes.

Worst case: O(n). If many keys collide, the algorithm may have to check a large number of positions.

3. Experimental Results:

dataset	n	structure	insert_ms	search_ms	delete_ms	insert_steps	search_steps	delete_steps
random	100	BST	0.004	0.006	0.006	6.9	7.9	5.2
random	100	Hash	0.019	0.002	0.002	3.9	1.3	1.3
random	1000	BST	0.092	0.095	0.118	10.5	11.5	9.3
random	1000	Hash	0.153	0.029	0.028	3.3	1.4	1.4
random	10000	BST	1.564	1.565	1.614	16.5	17.5	15.1
random	10000	Hash	2.794	0.324	0.293	4.5	1.2	1.2
sorted	100	BST	0.042	0.051	0.008	49.5	50.5	1.0
sorted	100	Hash	0.028	0.014	0.010	4.0	1.3	1.3
sorted	1000	BST	3.169	3.357	0.083	499.5	500.5	1.0
sorted	1000	Hash	0.230	0.146	0.100	3.7	1.5	1.5
sorted	10000	BST	194.166	189.084	0.074	4999.5	5000.5	1.0
sorted	10000	Hash	0.662	0.066	0.072	4.4	1.2	1.2

In my tests, BST was faster at inserting 100 random elements (0.004 ms vs 0.019 ms), probably because the hash table has extra overhead on small inputs. With 10,000 random elements, the hash table was faster at searching (0.324 ms vs 1.565 ms). The biggest difference was with 10,000 sorted keys: BST insertion took 194.166 ms vs 0.662 ms for the hash table. I think this is because a plain BST doesn't balance itself, so sorted keys turn it into a linked list and insertion becomes O(n). Overall, the hash table was more reliable, especially for sorted data.

4.Why Performance Differs in Practice:

Input order: BST becomes slower when I insert sorted numbers because the tree becomes unbalanced.

Collisions: In a Hash Table, different keys can go to the same place, so it sometimes needs more time to find them.

Java and computer performance: The results can also change because of memory, Java optimization, and other programs running at the same time.

5.Conclusion:

If you just need fast lookups and don't care about order, a hash table is usually the better choice. A BST makes more sense when you need your data kept sorted. One catch: a plain BST can get really slow if you feed it already-sorted data, while a hash table doesn't have that problem and tends to stay fast.